

BORDER OPTICS — POORI RESEARCH SAMJHO

**PDF 13 of 22 — Core Analysis: H3 Border Proximity +
Buffer/Multi-Year Robustness**

**h3_border_proximity_robustness.py, buffer_sensitivity.py,
multiyear_trend.py**

Sakshi ke liye, Sakshi ki apni research par. PDF 11 mein H1 null nikla, PDF 12 mein H4 bhi null nikla. Is PDF mein ek bilkul alag tarah ka test hai — H3 — aur ek interesting twist: is poore project mein har jagah fix hone ke baad result significant-se-null hua hai, lekin yahan ek jagah ULTA hua — ek null result, fix hone ke baad, significant ban gaya. Ye is study ka ek genuine, stress-tested surviving finding hai.

Iske saath do aur robustness files bhi hain — buffer_sensitivity.py (kya result buffer-radius ke arbitrary choice par depend karta hai?) aur multiyear_trend.py (sirf do points ke bajaye, teen saal — 2021, 2023, 2025 — ka trend dekh kar kya pata chalta hai?).

Shuru Karne Se Pehle — Ek Bilkul Alag Tarah Ka Sawaal

PDF 11 aur PDF 12 dono mein humne ek hi tarah ka sawaal poocha tha: 'kya treated villages badle?' (H1, akele) aur 'kya treated villages control villages se zyada badle?' (H4, DiD). Dono ka jawaab null nikla. H3 ek bilkul alag sawaal poochta hai — koi before/after ya treated/control comparison nahi, balki ek *gradient* ya 'dose-response' sawaal: 'agar VVP-I ka koi asar hai, toh kya wo asar us gaon ki border-se-doori ke hisaab se badalta hai?' Jaise — border ke bilkul paas wale gaon (jahan strategic priority sabse zyada hai) kya doosron se zyada badle, ya kam? Ye ek Spearman correlation test hai — PDF 3 mein hum correlation ka concept seekh chuke hain — jo `distance_to_border_km` (PDF 11 mein `compute_border_distance.py` se banayi gayi) ko har gaon ke NDBI-change/lights-change ke saath jodta hai.

Is PDF ki kahani baaki dono se ek badi wajah se alag hai: har jagah, is poore project mein, ek data-quality bug fix hone ke baad result significant se null hua hai (H1 ka 154-village bug, H4 ka contamination bug). H3 mein bilkul ulta hua — wahi Entry 22 ka same-day, complete re-extraction fix, jisne H1 aur H4 dono ko null bana diya, usi ne H3 ke summer-window NDBI-proximity result ko null se significant bana diya. Aur is baar, is naye result ko poori tarah stress-test kiya gaya — leave-one-district-out, randomization inference, aur spatial correction (jo PDF 16 mein detail mein aayega) — aur ye har test mein survive karta hai. Yahi is poore study ka sabse strong, genuinely-defensible surviving finding hai.

Teen Files, Ek Common Thread

`h3_border_proximity_robustness.py` H3 ka poora test aur uska stress-testing dono karta hai. `buffer_sensitivity.py` poochta hai — kya H1 ka result 500-meter buffer radius ke arbitrary choice par depend karta hai, ya 250m/1000m pe bhi wahi baat sach hai? `multiyear_trend.py` poochta hai — sirf 2021-vs-2025 (do points) dekhne ke bajaye, agar 2023 ka bhi data ho (teen points), toh kya ek deeper pattern dikhta hai jo do-point comparison chhupa deta hai? Teen files ka common thread ye hai — ye sab primary NDBI/lights result ko alag-alag angle se stress-test karte hain, taaki final verdict sirf ek hi tarike se check kiya gaya na ho.

Part A — `h3_border_proximity_robustness.py`

Setup — Imports Aur Constants

```
import json
```

```
import numpy as np
import pandas as pd
from scipy import stats
```

Purpose: json aur pandas/scipy.stats familiar hain. numpy (np) is file mein pehli baar seriously use hoti hai — ye Python ki sabse fundamental numerical-computing library hai, jisme khud pandas aur scipy bhi bane hain. Yahan ye random-shuffling (np.random) aur array-comparison (np.sign) ke liye use hogi. Syntax: import numpy as np — poore data-science ecosystem mein yehi standard nickname hai, jaise pandas ke liye pd.

```
DISTANCES = "data/processed/border_optics_master_villages_with_distance.csv"
RESULT_FILES = {
    "full_year": "data/processed/border_optics_village_results_analyzed.csv",
    "summer": "data/processed/border_optics_village_results_summer_analyzed.csv",
}
N_PERMUTATIONS = 2000
RNG_SEED = 42
OUT_JSON = "outputs/h3_robustness_results.json"
```

Purpose: DISTANCES wahi file hai jo PDF 11 mein compute_border_distance.py ne banayi thi — har gaon ki geodesic border-distance. N_PERMUTATIONS = 2000 aur RNG_SEED = 42 aage ke randomization-inference test ke liye hain — 2000 baar shuffle karna hai, aur seed fix karne se har baar exactly wahi 2000 random-shuffles milte hain, taaki result reproducible ho (koi aur isi seed se isi code ko chalaaye toh bilkul wahi numbers milein). Syntax: Simple constants, uppercase naming convention (Python mein convention hai ki module-level constants CAPS mein likhe jaate hain).

load_merged() Aur full_sample_rho() — Asli H3 Test

```
def load_merged(window):
    distances = pd.read_csv(DISTANCES)
    results = pd.read_csv(RESULT_FILES[window])
    merged = results.merge(
        distances[["village_id", "distance_to_border_km"]], on="village_id", how="inner"
    )
    return merged[merged["is_core_sample"] == True]

def full_sample_rho(window, metric):
    df = load_merged(window).dropna(subset=[metric, "distance_to_border_km"])
    rho, p = stats.spearmanr(df["distance_to_border_km"], df[metric])
    return rho, p, len(df)
```

Purpose: load_merged() distance-data aur NDBI/lights-change data ko village_id par merge karta hai, sirf core-sample villages rakhta hai — yahi setup wahi hai jo test_h3_border_proximity.py naam ki ek chhoti, simpler file mein bhi hai (wahi H3 ka

original, sabse pehla version — is robustness file ke andar `full_sample_rho()` function usi test ko phir se, khud, replicate karta hai, isliye humein H3 ka poora concept seedha isi ek file se seekhne ke liye kaafi hai). `full_sample_rho()` hi wo asli H3 test hai: distance aur change-metric ke beech Spearman correlation nikaalna. Rho positive ho toh matlab 'jitna door utna zyada change,' negative ho toh 'jitna paas utna zyada change.' Syntax: `df.merge(other[cols], on="key", how="inner")` — do dataframes ko ek common column ke through jodna, sirf wahi rows rakhna jo dono mein match karti hain ("inner" join).

EXAMPLE — H3 Ki History — Ek Reversal Jo Ulti Direction Mein Gaya

Development Log Entry 6 mein, jab ye test pehli baar likha gaya tha (us waqt border-distance UTM 44N projection se nikaali gayi thi, PDF 11 mein dekha), full-year lights-proximity already significant tha ($\rho \approx -0.259$), jabki summer NDBI-proximity null tha. Entry 18 mein jab UTM se geodesic distance mein switch hua (kyunki UTM 44N far-east villages ke liye distance ko systematically ghata/badha raha tha), sabhi H3 ke rho/p values thodi der se badle, lekin koi bhi significance flip nahi hui — full-year lights significant hi raha, baaki null hi rahe. Fir Entry 22 aaya — wahi same-day, complete summer-extraction fix jisne H1 aur H4 dono ko significant se null bana diya (PDF 11, PDF 12 mein dekha) — aur is baar summer NDBI-proximity ka rho 0.038 ($p = 0.641$, null) se seedha 0.291 ($p = 0.0000026$, significant) pe chala gaya. Ye poore is study ka EKLAUTA aisa case hai jahan ek genuine data-fix ne kisi result ko null se significant banaya, ulta nahi. Interesting baat ye hai ki lights-proximity, dono windows mein, is fix se untouched raha — full-year wala significant hi raha (kyunki wo pehle se hi UTM-se-geodesic switch ke baad se stable tha), aur summer wala null hi raha (rho -0.071, $p = 0.266$, dono fix se pehle aur baad mein exactly wahi).

`leave_one_district_out()` — Ek District Hataa Kar Dekho

```
def leave_one_district_out(window, metric, full_rho):
    df = load_merged(window)
    districts = sorted(df["district"].unique())
    rows = []
    for d in districts:
        sub = df[df["district"] != d].dropna(subset=[metric, "distance_to_border_km"])
        rho, p = stats.spearmanr(sub["distance_to_border_km"], sub[metric])
        rows.append({
            "dropped_district": d, "n": len(sub), "rho": float(rho), "p": float(p),
            "significant": bool(p < 0.05),
            "same_sign_as_full_sample": bool(np.sign(rho) == np.sign(full_rho)),
        })
```

Purpose: PDF 12 mein humne wild-cluster-bootstrap ka zikr suna tha (14 clusters ka concern) — leave-one-district-out isi concern ko ek seedhe, samajhne-mein-aasan tareeke se test karta hai: agar hum ek district ko पूरी तरह hata dein aur baaki 13 districts se hi rho dobara nikaalein, kya wahi answer aata hai? Agar sirf ek district

(jaise koi bada outlier district) poore result ko drive kar raha ho, toh use hataane se rho significantly badal jaayega ya sign hi palat jaayega. Loop har district ke liye ye check karta hai, 14 baar, aur har baar ka result ek dictionary mein store hota hai. Syntax: `sorted(df["col"].unique())` — column ki saari alag values nikaal kar alphabetically sort karna. `df[df.district != d]` — us ek district ko chhod kar baaki sab rakhna (`!=` 'not equal to'). `np.sign(x)` — number ka sign nikaalta hai: positive ho toh +1, negative ho toh -1, zero ho toh 0 — do numbers same direction mein hain ya nahi, ye check karne ka seedha tareeka.

```
n_sig = sum(r["significant"] for r in rows)
n_same_sign = sum(r["same_sign_as_full_sample"] for r in rows)
rhos = [r["rho"] for r in rows]
return {
    "rows": rows, "n_significant_of_14": n_sig, "n_same_sign_of_14": n_same_sign,
    "rho_min": min(rhos), "rho_max": max(rhos),
}
```

Purpose: 14 individual results ko summarize kiya jaata hai — kitni baar (14 mein se) result significant raha, kitni baar sign same raha, aur rho ki poori range (min se max) kya rahi. Syntax: `sum(condition for item in list)` — Python mein True/False ko 1/0 ki tarah treat kiya jaata hai, isliye ek generator-expression ke andar True values ginne ke liye seedha `sum()` use ho sakta hai — 'kitni baar True hua' poochne ka ek chhota, elegant tareeka. `min(list)/max(list)` — built-in functions, sabse chhoti/badi value nikaalte hain.

randomization_inference() — Bina Kisi Treatment-Label Ke Permutation Test

```
def randomization_inference(window, metric, full_rho):
    # permutation test on the correlation - shuffle one var, see how extreme
    # observed rho is. No treatment label here to reassign like H1/H4, so
    # this is just "could this rho happen from an unstructured shuffle."
    df = load_merged(window).dropna(subset=[metric, "distance_to_border_km"])
    x = df["distance_to_border_km"].values
    y = df[metric].values
    rng = np.random.RandomState(RNG_SEED)
    perm_rhos = np.empty(N_PERMUTATIONS)
    for i in range(N_PERMUTATIONS):
        perm_rhos[i] = stats.spearmanr(x, rng.permutation(y))[0]
```

Purpose: Comment khud explain karta hai ki ye H1/H4 ke randomization-inference se thoda alag hai — wahan 'treatment' label tha jise randomly reshuffle kiya ja sakta tha (koi village treated hai ya control, ye label shuffle karo). Yahan koi label nahi hai, sirf do continuous numbers hain (distance, aur change). Toh sawaal ye poocha jaata hai: agar in dono ke beech koi asli relationship na ho, sirf random coincidence se, kya itni badi rho value aa sakti thi? Isko test karne ke liye, y (change-metric) ko baar-baar

randomly shuffle kiya jaata hai — isse x (distance) aur y ka koi asli connection nahi rehta, sirf random pairing reh jaata hai — aur har shuffle ke baad phir se rho nikaala jaata hai. 2000 shuffles se 2000 'agar koi relationship na hota toh rho kaisa dikhta' wale numbers milte hain — ye ek poori null-distribution banate hain. Syntax: `.values` — pandas Series ko ek plain numpy array mein convert karna. `np.random.RandomState(seed)` — ek reproducible random-number-generator object banana (seed fix karne se same sequence har baar milegi). `np.empty(N)` — N khaali (garbage-value) slots ka array pehle se allocate karna, taaki loop mein baar-baar naya array banane ki zaroorat na pade — speed ke liye. `for i in range(N):` — N baar loop chalana. `rng.permutation(y)` — array y ka ek random shuffled version return karta hai (original y nahi badalta). `stats.spearmanr(x, y)[0]` — spearmanr do values (rho, p) ek tuple mein return karta hai, `[0]` se sirf rho chuna.

EXAMPLE — 2000 Shuffles Kyun, Aur Kya Ye Sach Mein Random Hai

Har shuffle mein `rng.permutation(y)` y ke values ko bilkul random order mein rakh deta hai — jaise agar 251 gaonon ka NDBI-change hai, toh wahi 251 numbers, lekin ab kisi random gaon ko assign ho gaye hain, us gaon ki asli distance-se koi lena-dena nahi raha. Agar asli data mein x aur y ka genuinely koi connection hai, toh asli (un-shuffled) rho in 2000 random-shuffle wale rho se zyada 'extreme' (bada ya chhota) hona chahiye — kyunki random shuffle ne toh wo connection tod diya hai. Agar asli rho in 2000 random-shuffles jaisa hi lagta hai, toh asli data mein bhi koi genuine connection nahi tha, sirf coincidence tha.

```
p_perm = (np.sum(np.abs(perm_rhos) >= np.abs(full_rho)) + 1) / (N_PERMUTATIONS + 1)
return {
    "n_permutations": N_PERMUTATIONS, "perm_rho_mean": float(perm_rhos.mean()),
    "perm_rho_sd": float(perm_rhos.std()), "p_randomization": float(p_perm),
}
```

Purpose: Ye line poori randomization-inference p -value nikaalti hai: 2000 shuffled rho mein se kitne, apni absolute value mein, asli rho (`full_rho`) jitne ya usse zyada extreme the — unko count karke, $(\text{count} + 1)$ ko $(2000 + 1)$ se divide kiya jaata hai. Agar sirf 1-2 shuffles hi asli rho jitne extreme aaye, toh p -value bahut chhoti hogi — matlab asli rho genuinely rare/unusual hai, coincidence se aana mushkil hai. Syntax: `np.abs(array)` — poore array ke har element ka absolute value (negative sign hata dena), taaki positive aur negative rho dono 'kitna extreme hai' ke hisaab se compare ho sakein, sirf direction ke hisaab se nahi. `>=` element-wise comparison, ek boolean array deta hai. `np.sum(boolean_array)` — kitne True hain, unhe gin leta hai.

EXAMPLE — Wo '+1' Kyun Hai — Ek Chhota Lekin Zaroori Detail

Formula mein $(... + 1) / (N_PERMUTATIONS + 1)$ likha hai, sirf $... / N_PERMUTATIONS$ nahi. Iski wajah hai: asli, un-shuffled data khud bhi ek valid 'possible arrangement' hai (2000 shuffles ke saath total 2001 possible arrangements ban jaate hain, asli ko milaakar). Agar sirf 2000 se divide karte, aur koi bhi shuffle asli rho se extreme na

aata, toh p-value $0/2000 = 0.0$ aata — jo statistically galat claim hai (kabhi bhi 'ye impossible hai' mat kaho, sirf 'humne itna try kiya aur nahi mila' keh sakte ho). '+1' dono numerator aur denominator mein jodne se p-value kabhi bilkul zero nahi aata (kam se kam $1/2001 \approx 0.0005$ hota hai) — ye is field mein ek standard, well-known convention hai (permutation-test literature mein), sirf ek arbitrary hack nahi.

main() — Chaar Combinations, Poora Test Aur Ek Integrity-Check Ki Kahani

```
def main():
    combos = [
        ("summer", "ndbi_change", "H3 NDBI-proximity, summer window (the flagged, unstressed finding)"),
        ("full_year", "lights_change", "H3 lights-proximity, full-year window (Holm-significant, NOT previously flagged for this check)"),
        ("full_year", "ndbi_change", "H3 NDBI-proximity, full-year window (already null -- checked anyway, for completeness)"),
        ("summer", "lights_change", "H3 lights-proximity, summer window (already null -- checked anyway, for completeness)"),
    ]
```

Purpose: Chaar combinations ki ek list — har ek ek tuple (window, metric, description). Descriptions khud bata rahe hain ki kaunsa combination kis wajah se test ho raha hai — do 'asli' interesting cases (summer NDBI ka naya finding, full-year lights ka pehle-se-significant finding) aur do 'completeness ke liye' null cases. **Syntax:** List of tuples — har tuple 3 items ka hai, jise loop mein 3 variables mein unpack kiya jaayega.

```
all_results = {}
for window, metric, description in combos:
    key = f"{window}_{metric}"
    rho, p, n = full_sample_rho(window, metric)
    ...
    loo = leave_one_district_out(window, metric, rho)
    ...
    rand = randomization_inference(window, metric, rho)
    ...
    all_results[key] = {
        "description": description, "full_sample": {"rho": rho, "p": p, "n": n},
        "leave_one_district_out": loo, "randomization_inference": rand,
    }

with open(OUT_JSON, "w") as f:
    json.dump(all_results, f, indent=2)
```

Purpose: for window, metric, description in combos: — har tuple ko teen variables mein tod kar loop chalana. Har combination ke liye, teeno function (full-sample rho, leave-one-out, randomization) chalte hain, aur results ek nested dictionary all_results

mein jama hote hain — key hai "summer_ndbi_change" jaisa kuch, value hai poora result-set. Aakhir mein poori dictionary JSON file mein save ho jaati hai — yahi outputs/h3_robustness_results.json hai. Syntax: f"{window}_{metric}" — do variables ko underscore se jod kar ek naya string banana, dictionary-key ke liye. Nested dictionary — dictionary ke andar dictionary, jismein loo aur rand khud dictionaries hain.

EXAMPLE — Entry 32 — Jab Ek 'RESOLVED' Claim Ke Peeche Koi Asli File Hi Nahi Thi

Ye poori file, code ki tarah, kaafi der se likhi ja chuki thi — lekin Development Log Entry 32 mein ek chhota, honest, thoda embarrassing gap pakda gaya: ANALYSIS_FREEZE.md already keh raha tha ki ye H3 stress-test 'RESOLVED' hai, aur specific LOO/randomization numbers bhi quote kar raha tha — lekin jis file mein wo numbers save hone chahiye the, outputs/h3_robustness_results.json, wo disk par exist hi nahi karti thi. Matlab ye claim, jab likha gaya tha, kisi actual-re-runnable proof se backed nahi tha. Entry 32 mein script ko asal mein, live, fresh data ke against chalaaya gaya — aur achi baat ye thi ki numbers exactly wahi nikle jo pehle se claim kiye gaye the (summer NDBI-proximity: 14/14 significant, rho range [+0.183, +0.337], randomization $p < 0.001$; full-year lights-proximity: 14/14 significant, rho range [-0.292, -0.175], randomization $p < 0.001$) — toh underlying claim sahi thi, bas usko back karne wala asli file missing tha. Ye is poore project ki ek core habit ko dikhata hai: sirf ek likha hua 'RESOLVED' word kaafi nahi hai — jab tak koi usse dobara, live data se, re-run karke verify na kare, wo claim sirf ek assertion hai, proof nahi.

Final State — H3 Ka Current, Stress-Tested Verdict

Ye hai outputs/h3_robustness_results.json se seedha liya gaya, current, chaaron-combination ka poora result:

Combination	Rho	p (raw)	LOO Significant	Randomization p
Summer NDBI-proximity	+0.291	2.64e-06	14/14	0.0005
Full-year Lights -proximity	-0.252	5.48e-05	14/14	0.0005
Full-year NDBI-proximity	+0.049	0.443 (null)	1/14	0.453
Summer Lights -proximity	-0.071	0.266 (null)	1/14	0.257

Dono significant combinations (summer NDBI, full-year lights) — 14 mein se 14 baar, koi bhi district hataao, sign aur significance same rehti hai, aur randomization inference bhi inhe genuinely-rare bataata hai ($p \approx 0.0005$, matlab 2000 shuffles mein se

ek bhi asli rho jitna extreme nahi aaya). Dono null combinations bilkul ek null jaisa hi behave karte hain (sirf 1/14 baar 'significant,' jo random-chance ke hisaab se bhi expected hai). PDF 16 mein ek aur, sabse strict layer aayega — spatial-autocorrelation correction (kyunki paas-paas ke gaon apne aap mein correlated hote hain, independent observations nahi) — jiske baad, ANALYSIS_FREEZE.md ke mutabik, summer NDBI-proximity ka p sirf 0.02949 tak (abhi bhi significant, lekin bahut kam margin se) aur full-year lights-proximity ka p 0.00350 tak (comfortably significant) aata hai. Yani dono surviving finding hain, lekin ek (summer NDBI) doosre (full-year lights) se zyada 'fragile' hai — is poori PDF ka job tha ye dikhana ki ye numbers exact kahan se, kaise compute hote hain.

Part B — buffer_sensitivity.py

Ye file poochti hai: H1 ka NDBI-change result, jo humesha 500m ke buffer radius (har gaon ke around ek 500-meter ka circle, jisme se satellite pixels average kiye jaate hain — PDF 1 mein buffer-radius ka concept seekha tha) se compute hota hai, kya agar buffer chhota (250m) ya bada (1000m) rakha jaaye, toh bhi wahi answer aata hai?

Setup Aur test_buffer()

```
BUFFERS = {
    250: "data/processed/border_optics_buffer250_summer.csv",
    500: "data/processed/border_optics_village_results_summer_analyzed.csv",
    1000: "data/processed/border_optics_buffer1000_summer.csv",
}
SUMMARY_OUT = "data/processed/border_optics_buffer_sensitivity_summary.json"
```

Purpose: Path-dictionary pattern, ab familiar — is baar key integer hai (250, 500, 1000), string nahi, kyunki buffer-radius ek number hai. 500m wali file wahi primary, main analysis wali file hai jo analyze_results.py ne banayi thi (PDF 11) — 250m/1000m alag extraction files hain (PDF 10, File 7 mein extraction dekhi thi). Syntax: Dictionary keys integers bhi ho sakti hain, strings hi zaroori nahi.

```
def test_buffer(buffer_m, path):
    df = pd.read_csv(path)
    core = df[df["is_core_sample"] == True].copy()

    results = {"buffer_m": buffer_m, "n_core": len(core)}
    for outcome in ["ndbi", "lights"]:
        valid = core.dropna(subset=[f"{outcome}_before", f"{outcome}_after"])
        w_stat, w_p = stats.wilcoxon(
            valid[f"{outcome}_after"], valid[f"{outcome}_before"], alternative="greater"
        )
        change = valid[f"{outcome}_after"] - valid[f"{outcome}_before"]
        results[f"{outcome}_n_valid"] = len(valid)
```

```

results[f"{outcome}_mean_change"] = change.mean()
results[f"{outcome}_median_change"] = change.median()
results[f"{outcome}_wilcoxon_stat"] = w_stat
results[f"{outcome}_wilcoxon_p"] = w_p

```

```

return results

```

Purpose: Wilcoxon test ka pattern bilkul PDF 11 ke `analyze_results.py` jaisa hai. Naya cheez hai — results dictionary ko dynamically build karna: ek khaali dictionary shuru hoti hai, aur loop ke andar, har outcome ("ndbi", "lights") ke liye, f-string se banaayi gayi keys (f"{outcome}_n_valid" = "ndbi_n_valid" ya "lights_n_valid") mein values daali jaati hain. Isse ek hi loop, ek hi dictionary mein, dono outcomes ke saare stats side-by-side store kar deta hai, bina alag-alag variables banaaye. Syntax: {"key": value} — dictionary literal syntax, seedhe values ke saath shuru karna. dict[f"..."] = value — dictionary mein ek naya, dynamically-named key add karna (agar key pehle se nahi hai, toh naya ban jaata hai).

matched_subsample_test() — Ek Purani Confound Ki History

```

def matched_subsample_test(common_ids, outcome="ndbi"):
    # 250m/1km ran later than 500m and the archive backfilled in between,
    # so restrict to villages valid at all three radii for a fair comparison
    print(f"\n=== Matched-subsample check ({outcome.upper()}), n={len(common_ids)} villages "
          f"valid at all three buffer radii ===")
    results = []
    for buffer_m, path in BUFFERS.items():
        df = pd.read_csv(path)
        sub = df[df["village_id"].isin(common_ids)].dropna(subset=[f"{outcome}_before",
f"{outcome}_after"])
        change = sub[f"{outcome}_after"] - sub[f"{outcome}_before"]
        w_stat, w_p = stats.wilcoxon(sub[f"{outcome}_after"], sub[f"{outcome}_before"],
alternative="greater")
        print(f" {buffer_m}>5d}m n={len(sub):3d} mean change={change.mean():+.5f}
Wilcoxon p={w_p:.6f}")
        results.append({
            "buffer_m": buffer_m, "outcome": outcome, "n": len(sub),
            "mean_change": change.mean(), "wilcoxon_p": w_p,
        })
    return results

```

Purpose: Comment khud is function ki poori wajah bata deta hai — ye function sirf un villages par test chalata hai jo teeno buffer radii (250/500/1000m) mein valid hain, taaki comparison 'fair' ho — har radius ka apna alag sample size na ho. Syntax: df["col"].isin(a_set) — column ki har value check karo ki wo kisi given set/list mein hai ya nahi, ek boolean mask deta hai jo filtering ke liye use hota hai. for k, v in

`dict.items()`: — dictionary ke har (key, value) pair par loop chalana.

EXAMPLE — Entry 12-13 — Ek Confound Jo Pehle Real Lagi, Fir Diagnose Hui

Jab ye buffer-sweep sabse pehle chalaaya gaya tha (Entry 13), pehla nateeja chinta-janak tha: 500m par result significant tha ($n=154$, $p<0.000001$), lekin 250m aur 1km par null ($n=251$ dono, $p=0.126$ aur 0.899) — jaise buffer-radius badalte hi result gaayab ho raha ho. Lekin likhne se pehle, wajah check ki gayi: 500m ke 97 gaon null the jo 250m/1km mein bilkul null nahi the — aur ek buffer-radius change se itna bada coverage-difference aana physically ajeeb hai (Sentinel-2 tiles dus-dus kilometer chaudhi hoti hain, ek gaon 250m ke andar aata hai ya 1km ke andar, isse scene-availability nahi badalni chahiye). Asli wajah simple thi — 250m/1km extraction 500m ke hafton baad chala tha, aur usi beech Sentinel-2 ka archive un tareekhон ke liye naye scenes backfill kar chuka tha (bilkul wahi 'archive backfill' pattern jo Entry 21 mein Sikkim ke case mein bhi mila tha). Fix: matched-subsample restrict karna — 154 villages jo teeno radii mein valid the, unhi par test chalaana — aur us par sabhi teen radii significant nikle (250m $p=0.000009$, 500m $p<0.000001$, 1km $p=0.001471$). Isko script aur paper dono mein khule taur pe likha gaya, chhupaaya nahi gaya.

```
def main():
    ...
    dfs = {b: pd.read_csv(p) for b, p in BUFFERS.items()}
    valid_sets = {}
    for b, df in dfs.items():
        core = df[df["is_core_sample"] == True]
        valid_sets[b] = set(core.dropna(subset=["ndbi_before", "ndbi_after"])["village_id"])
    common_ids = valid_sets[250] & valid_sets[500] & valid_sets[1000]
```

Purpose: Teeno buffer files ek dictionary-comprehension se ek hi line mein padh li jaati hain. Fir har buffer ke liye, un villages ki village_id ek Python set mein daali jaati hai jinke paas valid NDBI data hai. Aakhri line — & operator — teeno sets ka *intersection* nikaalti hai: sirf wo IDs jo teeno sets mein common hain, yaani wo villages jo teeno radii pe valid hain. Syntax: {key: expr for key, val in dict.items()} — dictionary comprehension, ek line mein poori nayi dictionary banana. set(list) — ek Python set banana (duplicate-free, unordered collection). set1 & set2 & set3 — set intersection operator, sirf teeno mein common items.

Final State — Buffer Sensitivity Ka Current Verdict

Entry 31 mein, jab sabhi teen buffer radii ko same-day re-extract kiya gaya (500m primary re-extraction ke turant baad hi 250m/1km bhi), archive-timing confound khatam ho gaya — ab teeno radii pe poora 251/251 core-sample valid hai, koi coverage-gap nahi, isliye matched_subsample aur as_extracted ab literally same numbers dete hain:

Buffer	NDBI Mean Change	NDBI Wilcoxon p	Lights Wilcoxon p
250m	+0.00175	0.12137	0.99998
500m	-0.00090	0.43583	0.99999
1000m	-0.00293	0.89761	1.00000

Teeno radii null hain — koi bhi buffer-size choice result ko significant nahi banati. Ek honest detail jo script khud disclose karta hai: NDBI ka mean-change teeno radii mein sign bhi consistent nahi rakhta — 250m par positive (+0.00175), 500m aur 1km par negative (-0.00090, -0.00293) — koi bhi significance ke paas nahi, lekin agar koi real, meaningful built-up-change effect hota, toh alag-alag buffer radii ko kam se kam direction pe toh agree karna chahiye tha. Ye ek additional, supporting evidence hai null-conclusion ke liye, koi complication nahi.

Part C — multiyear_trend.py

Ab tak humne hamesha do points dekhe hain — 2021 (before) aur 2025 (after). Ye file ek teesra point jodti hai — 2023 — aur poochti hai: kya sirf do points ke beech ek seedhi line khinchne se koi zaroori detail chhupi reh jaati hai?

Setup Aur per_village_slope()

```
import argparse
import json

import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
from scipy import stats

YEARS = [2021, 2023, 2025]
```

Purpose: Sabhi imports familiar hain (PDF 12 mein statsmodels dekha tha). YEARS ek list hai, teen saalon ki — is poori file ka core idea hai ki har gaon ke liye in teen saalon ka data hoga, aur ek line/slope fit hogi. **Syntax:** List literal, teen integers.

```
def per_village_slope(row, outcome):
    y = np.array(YEARS, dtype=float)
    vals = np.array([row[f"{outcome}_{yr}"] for yr in YEARS], dtype=float)
    if np.isnan(vals).any():
        return None, None
    slope, intercept, r, p, se = stats.linregress(y, vals)
    return slope, r ** 2
```

Purpose: Ek hi gaon (ek row) ke liye, teeno saal ka value nikaal kar (`f"{outcome}_{yr}"` se, jaise `ndbi_2021`, `ndbi_2023`, `ndbi_2025`), agar koi bhi saal missing (NaN) ho toh None, None return karke us gaon ko skip kar diya jaata hai. `stats.linregress()` — ek naya function, jo saal (x-axis) aur value (y-axis) ke beech best-fit seedhi line fit karta hai, bilkul jaise school mein 'line of best fit' seekha hota hai. Ye ek saath 5 values return karta hai — slope (line ki dhaal, per-year change), intercept (line kahan se shuru hoti hai), r (correlation coefficient), p (uski significance), aur se (slope ka standard error) — jismein se sirf slope aur r-squared (r ka square, batata hai line data ko kitni achi tarah fit karti hai) rakhe jaate hain. Syntax: `[row[f"..."] for yr in YEARS]` — list comprehension, teen values ki list banana, loop se. `np.array(list, dtype=float)` — Python list ko numpy array mein badalna, sabhi values ko float type mein force karna. `np.isnan(arr).any()` — array mein koi bhi NaN (missing) value hai ya nahi, check karta hai. `a, b, c, d, e = function(...)` — ek function jo 5 values return kare, unhe ek saath 5 variables mein unpack karna. `r ** 2` — `**` Python ka 'power' operator hai, r ka square nikaalta hai.

EXAMPLE — Sirf Teen Points Se Bhi Ek Line Kyun Fit Ki Jaa Sakti Hai

School mein 'line of best fit' do se zyada points ke liye seekhaya jaata hai — jitni bhi scatter points ho, ek aisi seedhi line dhoondi jaati hai jo un sabse 'sabse kam door' ho (technically: sum-of-squared-distances minimum karti ho). Teen points ke saath bhi yahi hota hai — bas line thodi kam 'certain' hoti hai (r-squared, jo batata hai line kitni achi fit hai, do points ke saath hamesha perfect 1.0 hota, teen points ke saath asli data ki 'straightness' ka sach dikhata hai). Isi wajah se `mean_trend_r2` is file mein ek genuinely useful number hai — agar wo bahut kam ho, toh matlab teen points ek seedhi line mein fit hi nahi hote, trend ka concept khud hi weak hai.

`analyze_outcome()` — Har Gaon Ke Liye Loop, Fir Poore Sample Ka Test

```
def analyze_outcome(df, outcome, window):
    slopes, r2s = [], []
    for _, row in df.iterrows():
        slope, r2 = per_village_slope(row, outcome)
        slopes.append(slope)
        r2s.append(r2)
    df = df.copy()
    df[f"{outcome}_slope"] = slopes
    df[f"{outcome}_trend_r2"] = r2s
```

Purpose: Ab tak is poore series mein humne dekha hai ki pandas operations hamesha 'vectorized' hoti hain — poore column par ek saath (jaise `df["ndbi_after"] - df["ndbi_before"]`, PDF 11), bina kisi explicit loop ke, jo tez hota hai. Yahan pehli baar `.iterrows()` dikh raha hai — ye row-by-row, ek-ek karke iterate karta hai, jo vectorized se dheema hota hai. Isko yahan use karna padta hai kyunki `per_village_slope()` ko ek

saath teen columns (ndbi_2021, _2023, _2025) ki zaroorat hai per-row, aur linregress() jaisa function poore column par ek saath nahi chal sakta (har gaon ki apni alag regression hai) — ye ek honest trade-off hai, speed ke bajaye correctness/clarify choose karna, jab data ka size (251 gaon) chhota ho toh ye dheema-pan matter nahi karta. Syntax: for _, row in df.iterrows(): — pandas dataframe ke har row par loop, _ se row-index (jo yahan use nahi hoti, isliye underscore se 'ignore karo' bataya jaata hai) aur row se poori row (Series ki tarah) milti hai. list.append(x) — khaali list mein items jodna, loop ke andar.

```
valid = df.dropna(subset=[f"{outcome}_slope"])
slope_vals = valid[f"{outcome}_slope"]

w_stat, w_p = stats.wilcoxon(slope_vals, alternative="greater")
```

Purpose: Yahan ek naya, important detail hai — stats.wilcoxon() ab tak humne hamesha DO arrays ke saath dekha hai (wilcoxon(after, before)) — 'paired difference' test. Yahan sirf EK array (slope_vals) diya gaya hai. Jab Wilcoxon ko sirf ek array milta hai, wo use ZERO ke against test karta hai — poochta hai 'kya ye slopes, ek group ki tarah, zero se significantly upar hain?' (zero slope ka matlab 'koi trend nahi'). Yehi wahi Wilcoxon function hai jo PDF 11 se milta-julta hai, bas is baar ek doosri calling-convention mein use ho raha hai. Syntax: stats.wilcoxon(single_array, alternative="greater") — one-sample mode, values ko zero ke against test karta hai.

```
d1 = valid[f"{outcome}_2023"] - valid[f"{outcome}_2021"]
d2 = valid[f"{outcome}_2025"] - valid[f"{outcome}_2023"]
w1_stat, w1_p = stats.wilcoxon(d1, alternative="greater")
w2_stat, w2_p = stats.wilcoxon(d2, alternative="greater")
```

Purpose: Poore 2021-2025 trend ke alaawa, do sub-periods bhi alag-alag test hote hain — 2021→2023 (d1) aur 2023→2025 (d2) — taaki pata chale ki change poore window mein uniformly spread hai, ya sirf ek half mein concentrated hai. Syntax: Vectorized subtraction, do baar — bilkul PDF 11 ke ndbi_change = ndbi_after - ndbi_before jaisa pattern, sirf yahan 2021/2023/2025 ke beech.

EXAMPLE — Do-Point Comparison Ek Poori Kahani Chhupa Deti Hai

Current, corrected 3-point data se ye pattern bilkul saaf dikhta hai — full-year NDBI ke liye: 2021→2023 mein mean change -0.0428 hai ($p \approx 1.0$, matlab 'increase hua' ye bilkul saabit nahi hota, ulta decline hi lag raha hai), lekin 2023→2025 mein mean change +0.0243 hai, $p = 0.000000845$ (highly significant increase). Poore 2021-2025 period ka pooled slope-test null hai ($p=1.0$) kyunki decline aur recovery roughly cancel ho jaate hain — lekin agar sirf 2021-vs-2025 ka do-point comparison dekha jaata (jo H1 test karta hai), toh ye poori 'pehle gira, fir badha' wali kahani bilkul chhup jaati. Summer window mein bhi bilkul yahi shape hai (2021→2023 change -0.0231, $p \approx 1.0$; 2023→2025 change +0.0222, $p=0.00000018$). Night-lights mein bhi same pattern hai, dono windows mein (full-year: 2021→2023 rise $p=0.0000022$, phir

2023→2025 flat $p=0.748$; summer: 2021→2023 gira $p\approx 1.0$, phir 2023→2025 uthaa $p=0.000000000017$) — teeno case mein, poora period ek seedha trend nahi tha, balki do alag phases the jo teesra point (2023) dikhaaye bina kabhi nahi pata chalta.

```
long_rows = []
for _, row in valid.iterrows():
    for yr in YEARS:
        long_rows.append({"village_id": row["village_id"], "year": yr, "value":
row[f"{outcome}_{yr}"]})
long = pd.DataFrame(long_rows)
long["year_c"] = long["year"] - 2021 # centered, so the intercept is interpretable
model = smf.ols(f"value ~ year_c + C(village_id)", data=long).fit(
    cov_type="cluster", cov_kwds={"groups": long["village_id"]}
)
panel_coef = model.params["year_c"]
panel_p = model.pvalues["year_c"]
```

Purpose: Ye ek doosra, cross-check specification hai — PDF 12 mein `did_model.py` ka wide-se-long panel transformation yaad hoga; yahan bhi bilkul wahi idea hai, is baar district ki jagah `village_id` khud ek fixed-effect ban jaata hai (`C(village_id)` — 251 alag intercepts, har gaon ka apna). Formula "`value ~ year_c + C(village_id)`" ka matlab hai — har gaon ka apna baseline-level do (village FE), aur sirf `year_c` (saal, 2021 se centered) ka coefficient batayega ki, in-baseline-hote-hue, per-year kitna badlaav hai. Ye per-village-slope Wilcoxon test se ek bilkul alag statistical design hai (panel regression vs. simple signed-rank test), lekin agar dono ka jawaab match kare, toh confidence badhti hai ki result kisi ek particular technique ka artifact nahi hai. Syntax: `long_rows.append({...})` — nested loop (har gaon ke andar, har saal ke liye) se ek-ek dictionary banaakar list mein jodna, phir `pd.DataFrame(list_of_dicts)` se poora dataframe ek saath banana. `col - constant` — poore column se ek fixed number ghata dena (centering). `model.params["name"]/.pvalues["name"]` — PDF 12 se familiar pattern, fitted-model se ek specific coefficient ka value/p nikaalna.

EXAMPLE — Year Ko Centre Karna Kyun Zaroori Hai

`year_c = year - 2021` se 2021 ka `year_c` 0 ban jaata hai, 2023 ka 2, 2025 ka 4. Regression ka intercept (jo har village-fixed-effect ke through bhi implicitly represent hota hai) 'jab `year_c` zero ho tab value kya hai' batata hai — agar centering na karte, aur seedhe 2021/2023/2025 use karte, toh intercept ka matlab hota 'year 0 (yaani calendar ke bilkul shuru) mein value kya hogi' — ek number jiska koi practical matlab nahi. Centering se intercept ka matlab ban jaata hai 'is study ke baseline year (2021) mein value kya thi' — ek interpretable number.

main() — Familiar Pattern, Do Outcomes, Do Files Save

```
def main():
    parser = argparse.ArgumentParser(description="Fit a 2021/2023/2025 trend per village
and test its significance.")
```

```

parser.add_argument("--window", choices=["full_year", "summer"], required=True)
args = parser.parse_args()
window = args.window

df = pd.read_csv(DATA_PATHS[window])
core = df[df["is_core_sample"] == True].copy()

results = []
for outcome in ["ndbi", "lights"]:
    result, core = analyze_outcome(core, outcome, window)
    results.append(result)

core.to_csv(SLOPES_OUT[window], index=False)
with open(SUMMARY_OUT[window], "w") as f:
    json.dump(results, f, indent=2)

```

Purpose: Bilkul familiar argparse/loop/save pattern — bas ek chhota interesting detail: result, core = analyze_outcome(core, outcome, window) — analyze_outcome() function do cheezein return karta hai (result-dictionary, aur updated core dataframe jisme naye slope/r2 columns jud chuke hain), aur loop ke har iteration mein core ko dobara update kiya jaata hai — taaki dusri iteration (lights) ke slope-columns bhi usi dataframe mein add hon, pehli iteration (ndbi) ke columns ke saath-saath, na ki alag-alag dataframes mein. Syntax: Function se do values return hoke seedhe do variables mein assign hona — a, b = function().

Final State — Multi-Year Trend Ka Current Verdict

border_optics_multiyear_summary_fullyear.json/_summer.json se seedha:

Window	Outcome	Pooled Slope-Test p	2021→23 p	2023→25 p
Full-year	NDBI	1.00000 (null)	1.00000	8.45e-07
Full-year	Lights	0.04999 (borderline)	2.18e-06	0.74846
Summer	NDBI	0.44234 (null)	1.00000	1.82e-07
Summer	Lights	0.99999 (null)	1.00000	1.73e-11

Ek genuinely interesting, real coincidence: summer lights ka pooled slope-test p (0.9999941130208815) EXACT wahi hai jo buffer_sensitivity.py ke 500m summer lights Wilcoxon test se aaya tha, digit-by-digit. Ye coincidence nahi hai, ek genuine mathematical fact hai — teen equally-spaced points (2021, 2023, 2025) ke liye, OLS slope sirf pehle aur aakhri point ke antar par depend karta hai, beech wala point (2023) apne aap cancel ho jaata hai us formula mein. Isi wajah se 'slope vs zero' test aur

'seedha 2025-minus-2021 vs zero' test, jab dono ek hi villages ke set par chalte hain, exactly wahi Wilcoxon rank-order dete hain — full-year NDBI ka pooled slope-test p bhi (1.00000) PDF 11 ke H1 test ke full-year NDBI p (1.000000) se match karta hai, isi wajah se.

Poori PDF Ka Synthesis

Is PDF mein humne dekha: `h3_border_proximity_robustness.py` distance-se-change ka Spearman correlation nikaalta hai (`full_sample_rho`), phir usi result ko leave-one-district-out (14 baar, ek-ek district hataakar) aur randomization inference (2000 baar y ko shuffle karke) se stress-test karta hai; is poore study ka ek genuine, ulta-reversal yahin hai — Entry 22 ke fix ne summer NDBI-border-proximity ko null se significant banaya ($p=0.038 \rightarrow 0.291$), aur Entry 32 mein pata chala ki is claim ko back karne wali JSON file khud missing thi, jab tak usse dobara live-run karke verify nahi kiya gaya. `buffer_sensitivity.py` ne dikhaya ki NDBI ka null result 250m/500m/1000m teeno buffer radii pe consistent hai — Entry 12-13 ka ek archive-timing confound pehle isse chhupa raha tha, jab tak Entry 31 ke same-day re-extraction ne usse hamesha ke liye band nahi kar diya. `multiyear_trend.py` ne dikhaya ki 2021-2025 ka do-point comparison ek 'pehle gira, phir badha' wali do-phase kahani chhupaata hai jo sirf teesra point (2023) dikha sakta hai — aur ek chhota mathematical fact bhi seekha: teen equal-spaced points ka slope-test aur endpoint-difference test, ek hi villages ke against, exactly same Wilcoxon answer dete hain.

Agla PDF (PDF 14) wild-cluster-bootstrap dekhega — H4 ke 14-cluster-thin-hone wale concern ko sabse rigorous, computationally-heaviest tarike se test karne wala method, $2^{14} = 16,384$ sign-combinations ka poora enumeration. Jab bhi tayyar ho, bata dena.